Nodes & Tool Nodes

Overview

- Nodes can be:
  - ✅ **LLM calls** (e.g., calling an OpenAI model to generate text)
  - ✅ **Tool invocations** (e.g., Python functions, API calls, database queries)
  - ✅ **Control flow** elements (e.g., branching, loops, conditional routing)
  - ✅ **State transformers** (e.g., reading/updating graph state)
  - ✅ **Subgraphs** (a node can itself be a nested graph)
- **Best practice**: split up each logical step into it's own node rather than combining them into one
  - Checkpointing happens at each node so if all in one then we risk more back and forth due to failure
  - Makes it easy to swap out nodes for iterating
- If a node contains multiple operations, you may find it easier to convert each operation into a **task** rather than refactor the operations into individual nodes.
  - So instead of checkpointing per superstep we would now checkpoint per task to enhance durability incase one part of a nod fails we do not need to restart the whole node

```
@task
def _make_request(url: str):
    """Make a request."""
    return requests.get(url).text[:100]
```

Tool Nodes

- A `ToolNode` in LangGraph is a dedicated node whose only job is to execute tools.
- It is best practice to use a tool node instead of just letting the LLM call tools within a node as it improves control, observability, error handling, retries...

Implementation

1. **Define Tool Functions**
    - This should contain what the tool should do if called
    - We should add `@tool` above any tool functions since this lets us provide custom details provided to the LLM about how to call the tool:

	Default (no @tool)	**How to Customise What LLM Sees**
Tool name	Uses the Python function name `multiply` → `"multiply"`	Set explicitly: `@tool(name="multiply_numbers")`
Tool description	Uses the function docstring (first line or full docstring)	Supply custom description via docstring OR override with `@tool(description="...")`

Tool Arguments	**Names:** Taken directly from tool function inputs
	
	Types: Taken directly from tool function input type hints
	
	Descriptions: Not provided
		[Option 1] Customise Only Tool Argument Description: Use `@tool(parse_docstring=True)` and provide `Args:` section in docstring.
		
		[Option 2] Customise Tool Argument Names, Types & Descriptions: Define a Pydantic `args_schema`:
		
		`class MultiplyArgs(BaseModel):`
		`a: int = Field(description="The left operand")`
		`b: int = Field(description="The right operand")`
		
		`@tool(name="multiply_custom", args_schema=MultiplyArgs)`
		`def multiply(args: MultiplyArgs) -> int:`

2. **Bind Tools Schema to LLM**
3. **Pass the Same Tools to ToolNode**

```
# 1 Define Tool Function
@tool(name="multiply_numbers", parse_docstring=True)
def multiply(args: MultiplyArgs) → int:
    """Multiply two integers.

    Args:
        a: The first integer.
        b: The second integer.
    """
    return a * b


# 2 Bind Tools Schema to LLM
tools = [multiply]
llm_with_tools = llm.bind_tools(tools)

# 3 Pass the Same Tools to ToolNode
tool_node = ToolNode(tools)
```

What Does `ToolNode` Do?

- Takes your tools and stores them as a name→function mapping.
- When the LLM outputs a tool call, the graph routes to the ToolNode.
- The ToolNode extracts the call, matches the tool name, parses the arguments, runs the function, and returns a `ToolMessage` back into the state.
- Supports multiple tool calls, loops, and clean tracing.